

LAB MANUAL

Experiment: Understanding the Use of Constructors in Java

4. Theory Overview

A **constructor** is a special member function of a class that:

- Has the **same name as the class**
- Has **no return type**
- Is called **automatically** when an object is created

Why Constructors Are Needed

- To initialize object data
 - To ensure objects are created in a valid state
 - To avoid uninitialized or inconsistent objects
-

5. Experiment 1: Object Creation Without Constructor

Program

```
class Student {  
    int id;  
    String name;  
  
    void display() {  
        System.out.println(id + " " + name);  
    }  
}  
  
public class Test1 {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.display();  
    }  
}
```

Explain what you observe

6. Experiment 2: Default Constructor

Program

```
class Student {  
    int id;  
    String name;  
  
    Student() {  
        id = 101;  
        name = "Asha";  
    }  
  
    void display() {  
        System.out.println(id + " " + name);  
    }  
}  
  
public class Test2 {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.display();  
    }  
}
```

Explain your observation in technical words.

7. Experiment 3: Parameterized Constructor

Program

```
class Student {  
    int id;  
    String name;  
  
    Student(int i, String n) {  
        id = i;  
        name = n;  
    }  
  
    void display() {  
        System.out.println(id + " " + name);  
    }  
}  
  
public class Test3 {  
    public static void main(String[] args) {  
        Student s1 = new Student(102, "Ravi");  
        Student s2 = new Student(103, "Meera");  
        s1.display();  
        s2.display();  
    }  
}
```

8. Experiment 4: Constructor Overloading

Program

```
class Book {  
    int id;  
    String title;  
  
    Book() {  
        id = 0;  
        title = "Not Assigned";  
    }  
  
    Book(int i, String t) {  
        id = i;  
        title = t;  
    }  
  
    void display() {  
        System.out.println(id + " " + title);  
    }  
}  
  
public class Test4 {  
    public static void main(String[] args) {  
        Book b1 = new Book();  
        Book b2 = new Book(201, "Java Basics");  
        b1.display();  
        b2.display();  
    }  
}
```

9. Experiment 5: Constructor vs Method

Program

```
class Employee {  
    int id;  
  
    Employee(int i) {  
        id = i;  
        System.out.println("Constructor called");  
    }  
  
    void setId(int i) {  
        id = i;  
        System.out.println("Method called");  
    }  
}  
  
public class Test5 {  
    public static void main(String[] args) {  
        Employee e1 = new Employee(10);  
        e1.setId(20);  
    }  
}
```

10. Experiment 6: Multiple Objects – Constructor Called Each Time

Program

```
class Counter {  
    Counter() {  
        System.out.println("Object created");  
    }  
}  
  
public class Test6 {  
    public static void main(String[] args) {  
        Counter c1 = new Counter();  
        Counter c2 = new Counter();  
        Counter c3 = new Counter();  
    }  
}
```

11. Experiment 7: Constructor Ensuring Valid Object State

Program

```
class Product {  
    int price;  
  
    Product(int p) {  
        if (p > 0)  
            price = p;  
        else  
            price = 1;  
    }  
}
```

```
void display() {  
    System.out.println("Price = " + price);  
}  
}
```

```
public class Test7 {  
    public static void main(String[] args) {  
        Product p1 = new Product(100);  
        Product p2 = new Product(-50);  
        p1.display();  
        p2.display();  
    }  
}
```

12. Experiment 8: Constructor Chaining (Using this)

Program

```
class Demo {  
    Demo() {  
        this(10);  
        System.out.println("Default constructor");  
    }  
  
    Demo(int x) {  
        System.out.println("Parameterized constructor: " + x);  
    }  
}  
  
public class Test8 {  
    public static void main(String[] args) {  
        Demo d = new Demo();  
    }  
}
```

```
}  
}
```

Observation

- One constructor can call another
 - Avoids code duplication
-

13. Experiment 9: Constructor Access Control

Program

```
class Secure {  
    private Secure() {  
        System.out.println("Private constructor");  
    }  
  
    static Secure createObject() {  
        return new Secure();  
    }  
}  
  
public class Test9 {  
    public static void main(String[] args) {  
        Secure s = Secure.createObject();  
    }  
}
```


15. Final Task

Task Description

Design a class LibraryBook with:

- bookId, title, availability
- Constructor to initialize all values
- Prevent creation of a book without ID or title

Students must submit:

1. Java program
 2. Output
 3. Written explanation (5–6 lines) on why constructor is necessary
-

16. Viva in your own words answer.

Basic Level

1. What is a constructor?
2. Why does a constructor not have a return type?
3. When is a constructor executed?

Intermediate Level

4. What problem occurs if constructors are not used?
5. How is constructor overloading different from method overloading?
6. Why is constructor execution automatic but method execution manual?

Advanced / Conceptual Level

7. How do constructors help maintain object consistency?
8. Why would you make a constructor private?
9. Explain constructor chaining with an example.
10. Can a constructor call another constructor? How?